ITECH3102 DATA VISUALIZATION: COMPREHENSIVE LAB REPORT

Labs 1 to 4: Tableau, HTML, CSS, JS, SVG & D3

Name: ClientX

Student ID: StudentIDX

Course: ITECH3102

Date: 16/08/2026

Table of Contents

CHAPTER 1: INTRODUCTION OF TABLEAU SOFTWARE 2

1.1 Project 1: Installation of Tableau 2

1.1 Tableau Public Installation 2

1.2 Tableau Reader Installation 2

1.2 Project 2: Understand Tableau User Interface 3

1.2.1 Understand User Interface of Tableau 3

1.3 Project 3: Getting Data into Tableau 3

1.4 Project 4: Getting Data out of Tableau 5

1.5 Project 5: Getting Data out of Tableau 2 5

1.6 Project 6: Answering Questions 7

CHAPTER 2: FUNDAMENTALS OF WEB TECHNOLOGY: HTML, CSS 8

2.1 Project 1: HTML 8

2.2 Project 2: Using Browser's Built-in Developer Tools 8

2.3 Project 3: Cascading Style Sheets (CSS) 9

2.3.1 Embedding CSS in HTML 9

2.3.2 Using External CSS 10

2.4 Project 4: Answering Questions 11

2.4.1 Comment on "Perception in Visualization" 11

2.4.2 Examples of Data Visualization Encoding Based on Perception Principles 11

CHAPTER 3: JAVASCRIPT 12

3.1 Project 1: JavaScript 12

3.2 Project 2: Variables, Arrays, Objects 12

3.3 Project 3: Basic control structures of JavaScript 13

3.3.1 Selection Structure (if-else) 13

3.3.2 Repetition Structures (for and while) 14

3.3.3 User-defined Function 16

3.4 Project 4: Answering Questions 16

3.4.1 What is data abstraction? Give one or two examples of data abstraction. 16

3.4.2 Why do we need to derive data from raw data? 17

CHAPTER 4: SCALABLE VECTOR GRAPHICS (SVG) AND D3 18

4.1 Project 1: Scalable Vector Graphics (SVG) 18

4.2 Project 2: Styling SVG Elements, Layering and Transparency 21

4.3 Project 3: Setting up a D3 Development Environment 22

4.4 Project 4: Answering Questions 24

4.4.1 Based on concepts of the marks and channels, provide some examples of appropriate and inappropriate encoding choices. 24

4.4.2 Take an example of good visualizations (the graph presented in the topic 1, Napoleon's Army During the 1812, Visualization created by Charles Joseph Minard), and explain what marks and channel are used in the visualization, and discuss why they are used for? 24

Chapter 1: Introduction of Tableau Software

1.1 Project 1: Installation of Tableau

1.1 Tableau Public Installation

An error message, **"Oops, something went wrong. Please try again later,"** appeared when accessing the Tableau Public download page.

Therefore, the Tableau Public website was accessed to download and install **Tableau Desktop Public Edition**.

Figure 1.1: Tableau Public Desktop Edition successfully opened



Result: Tableau Public Desktop Edition was successfully installed and opened on the computer.

1.2 Tableau Reader Installation

Tableau Reader was downloaded from the official Tableau website and successfully installed. The application was then opened to verify the installation.

Figure 1.2: Tableau Reader Desktop Edition successfully opened



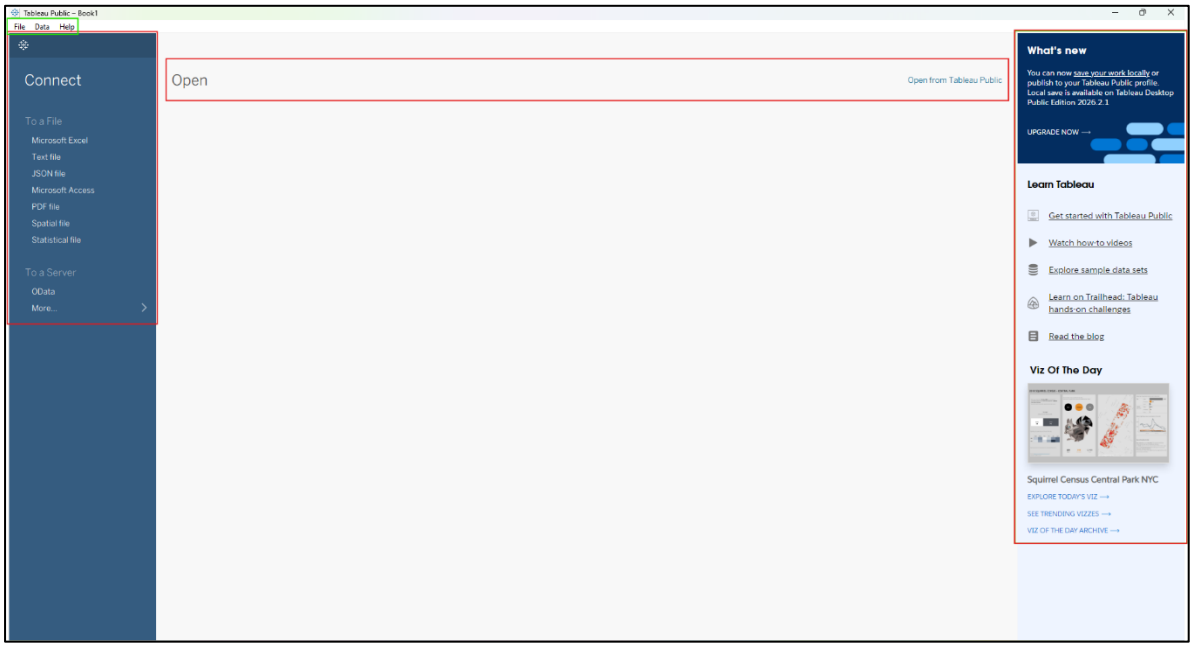
Result: Tableau Reader was successfully installed and opened.

1.2 Project 2: Understand Tableau User Interface

1.2.1 Understand User Interface of Tableau

Tableau Public was opened to explore its main user interface. The **Connect, Open, and Discover** sections of the Start Page were identified and examined.

Figure 2.1: Tableau Public Start Page



Observation: The Tableau Public Start Page provides options for connecting to data, opening existing workbooks, and accessing Tableau learning and discovery resources.

Result: The main sections of the Tableau Public Start Page were successfully identified and understood.

1.3 Project 3: Getting Data into Tableau

Procedure: From the Moodle Week 1 page, the dataset file named simple-data-boroughs.xlsx was downloaded and saved on the computer. Tableau Public was opened, and on the start page, the Microsoft Excel option located under the Connect pane was clicked. The downloaded Excel file was located and selected in the file explorer window. Tableau immediately loaded the data and opened the data source screen, showing a preview of the six columns (Borough, County, State, Population, Area sq-mi, Year). After confirming the data preview was correct, the Go to Worksheet tab at the bottom-left was clicked to enter the main visual workspace.

Observation & Result: After moving to the worksheet, the left-hand Data Pane was examined. Tableau automatically classified the fields into two groups. Borough, County,

and State appeared as blue Dimensions (categorical data). Population, Area sq-mi, and Year appeared as green Measures (quantitative data). Additionally, a globe symbol was present next to the geographic fields, and the # symbol was visible next to the numerical fields. This confirmed that Tableau successfully identified the data types.

Figure 3.1: Excel file connected and data preview

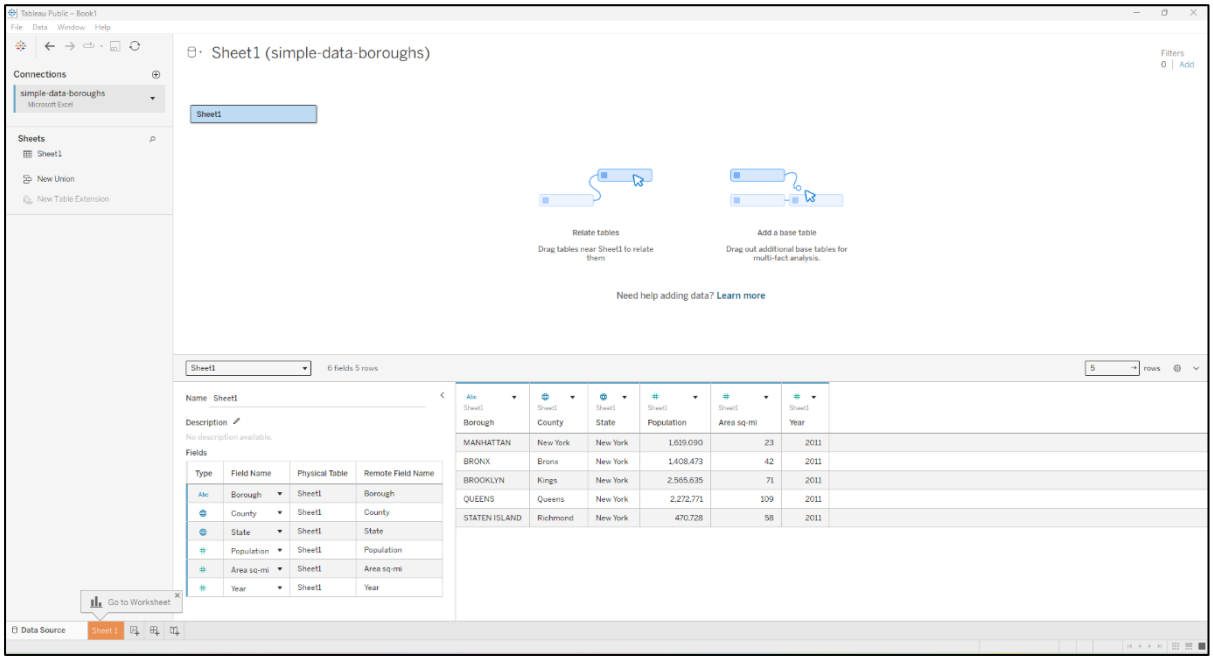
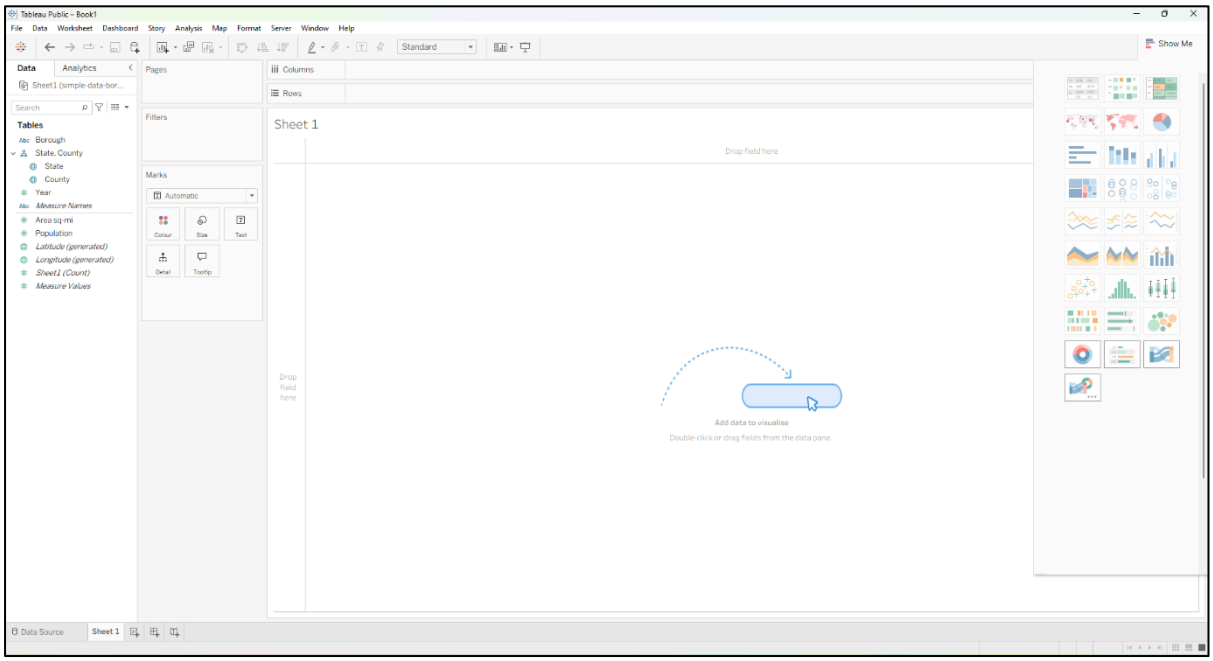


Figure 3.2: Main workspace showing Dimensions and Measures

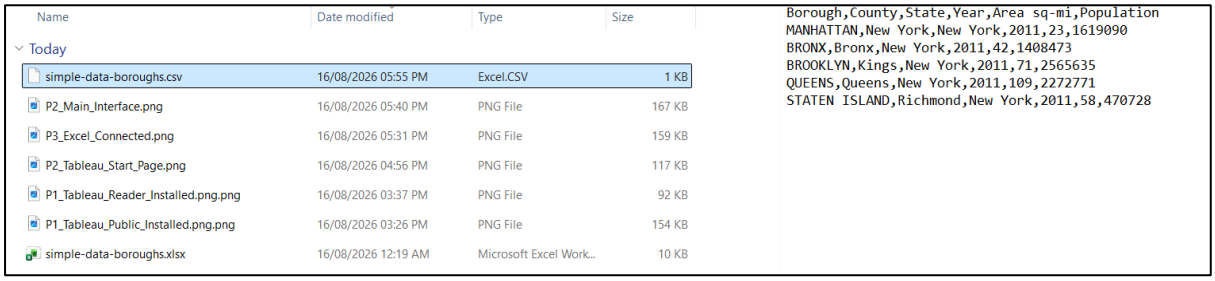


1.4 Project 4: Getting Data out of Tableau

Procedure: To export the connected data into a new file format, the Data menu located on the top toolbar of the Tableau worksheet was clicked. From the drop-down menu, the active data source Sheet1 (simple-data-boroughs) was selected, followed by clicking the Export Data to CSV option. A pop-up window appeared allowing me to choose a save location. The folder Assignment was navigated to, and the file was saved with the default name Sheet1 (simple-data-boroughs).csv.

Result: The data was successfully extracted from Tableau and converted into a .csv (Comma Separated Values) format file. This file is now saved on the local computer and can be opened and viewed in spreadsheet applications like Microsoft Excel or Notepad.

Figure 4.1: CSV file successfully exported in the folder



1.5 Project 5: Getting Data out of Tableau 2

Procedure: A bar chart was created to demonstrate a simple visualization.
The Population measure was dragged and dropped onto the Columns shelf. Next, the Borough dimension was dragged and dropped onto the Rows shelf. This generated five horizontal bars representing the population of each borough.
To save the work, the File menu was clicked, and the Save to Tableau Public... option was selected. I signed into my Tableau Public account and provided the title ITECH3102 Lab 1 - Borough Data for the workbook. After saving, the webpage view of the project was opened, and the Download button was clicked. From the dropdown menu, Tableau Workbook (.twbx) was selected, and the file was saved locally.

Result: The final bar chart was successfully created. The project was published to my Tableau Public profile and successfully downloaded as a packaged Tableau Workbook (.twbx file). This .twbx file can now be opened and viewed offline using the Tableau Reader application.

Figure 5.1: Final Bar Chart

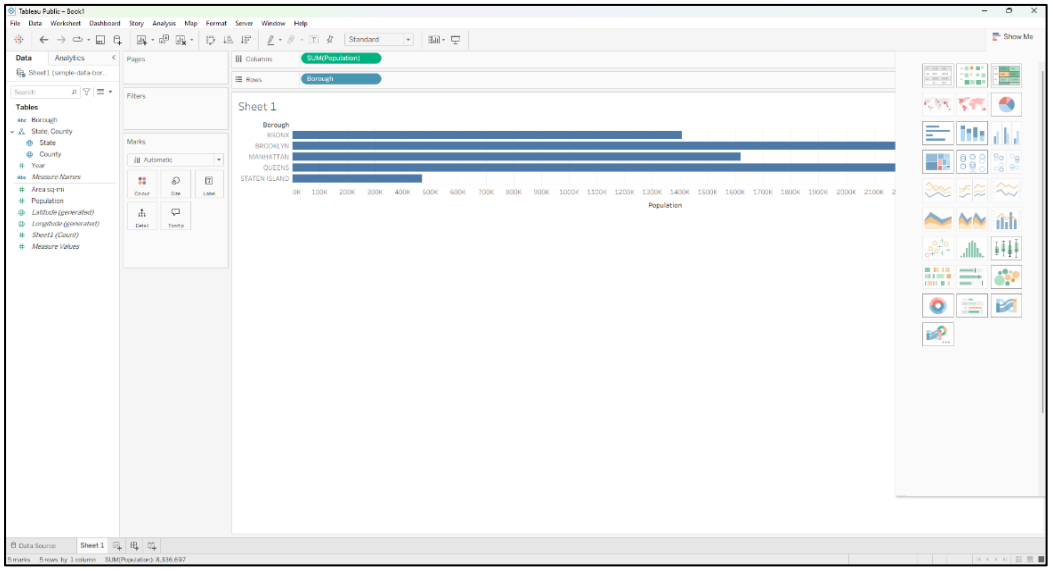


Figure 5.2: Project successfully saved on Tableau Public profile

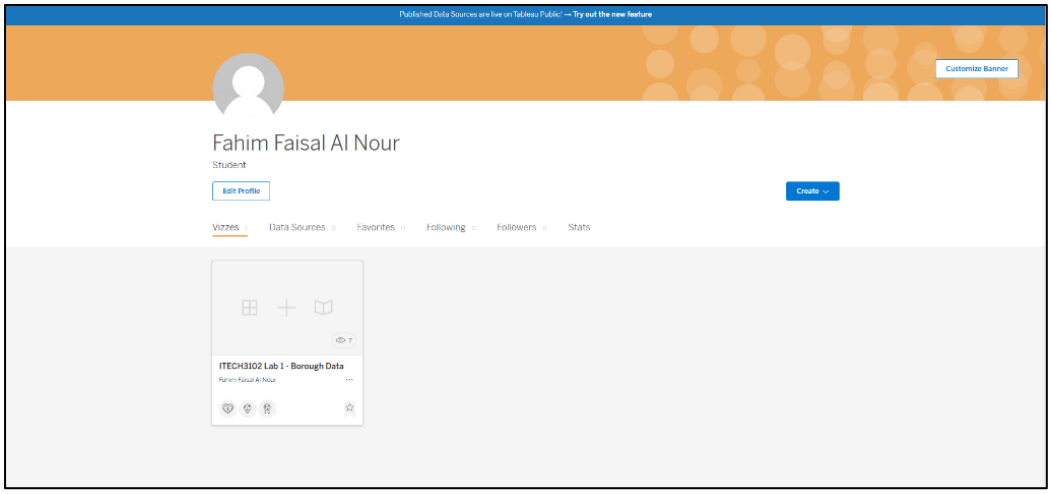
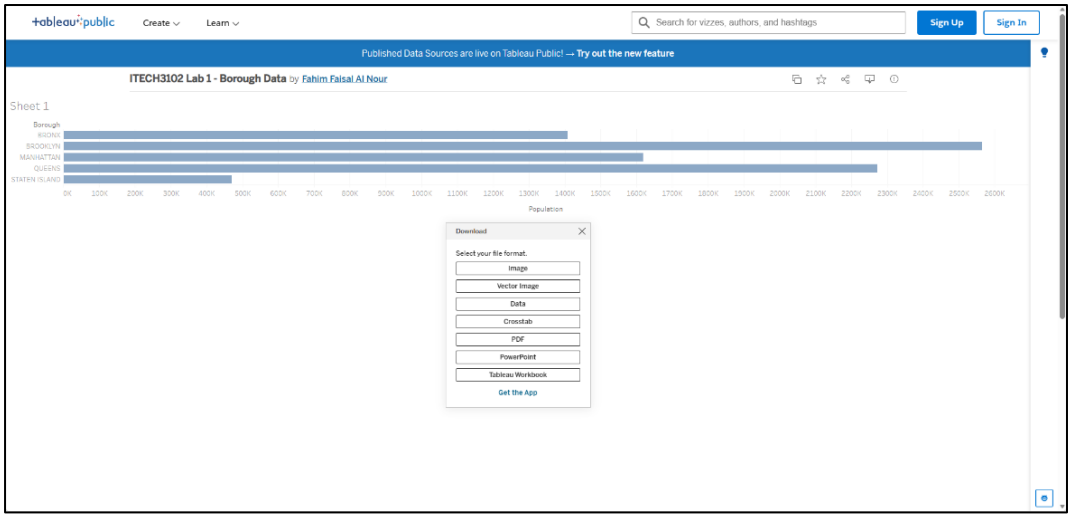


Figure 5.3: Downloaded Tableau Workbook (.twbx) file

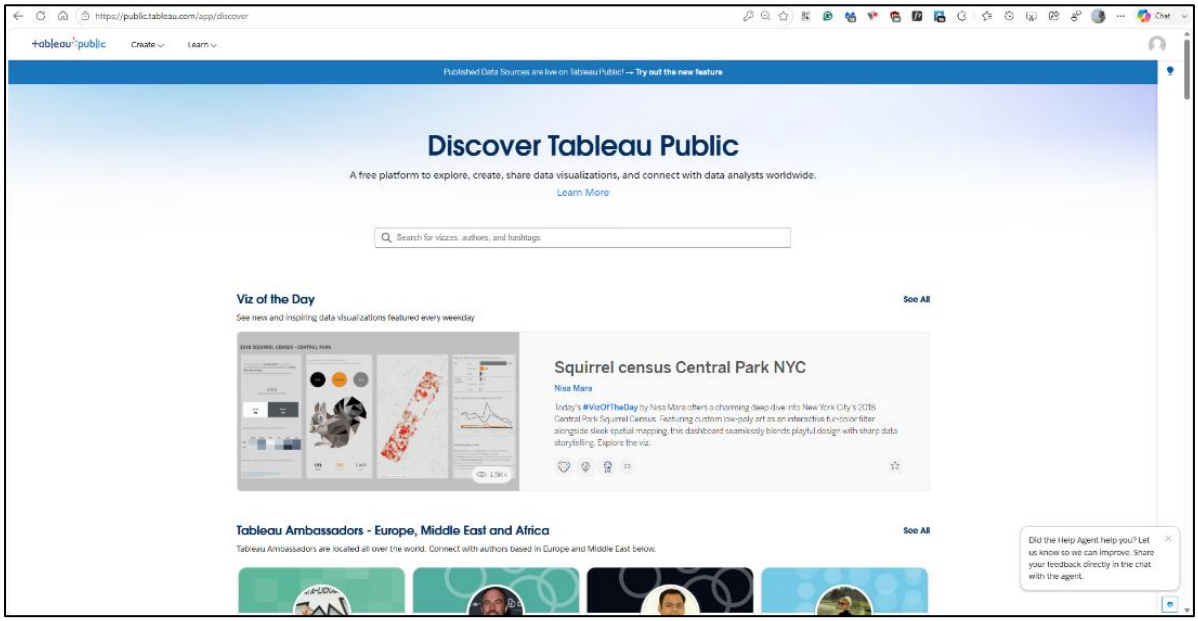


1.6 Project 6: Answering Questions

Procedure: The specific TED Talk video titled *"The truth about HIV"* by Hans Rosling was accessed via the link provided in the lab sheet. The video was watched thoroughly to understand how he uses data visualization. Additionally, the Discover Pane on the right side of the Tableau Public start page was explored to browse through the available online learning resources, examples, and training videos.

Observation & Result: Hans Rosling's talk was highly effective in demonstrating how interactive data can break down global health stereotypes. It utilized animated charts to show HIV as a global issue rather than a localized one, while highlighting that successful treatment efforts have drastically lowered death rates. Meanwhile, exploring the Discover Pane showed that Tableau provides a rich set of tutorials, forums, and sample workbooks which are excellent for self-learning.

Figure 6.1: Tableau Discover Pane showing learning resources



Chapter 2: Fundamentals of Web Technology: HTML, CSS

2.1 Project 1: HTML

Procedure: A new folder was created for the lab exercises. Using the Notepad++ text editor, a new file named firstHtml.html was created. The basic HTML structure, including <!DOCTYPE html>, <html>, <head>, and <body> tags, was typed into the file. Inside the body, a simple paragraph Welcome to my first web page was added. The file was saved with the .html extension.

Result/Observation: The firstHtml.html file was opened using the Google Chrome browser. The text "Welcome to my first web page" was successfully displayed. The browser's address bar showed the local file path beginning with file:///, confirming that the file was opened locally without a web server.

Figure 2.1: HTML source code

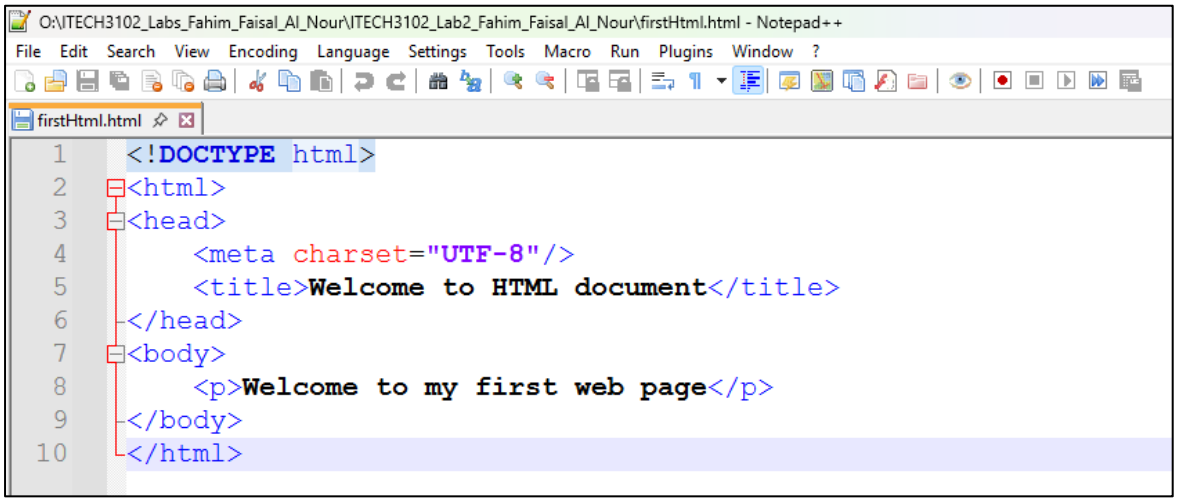
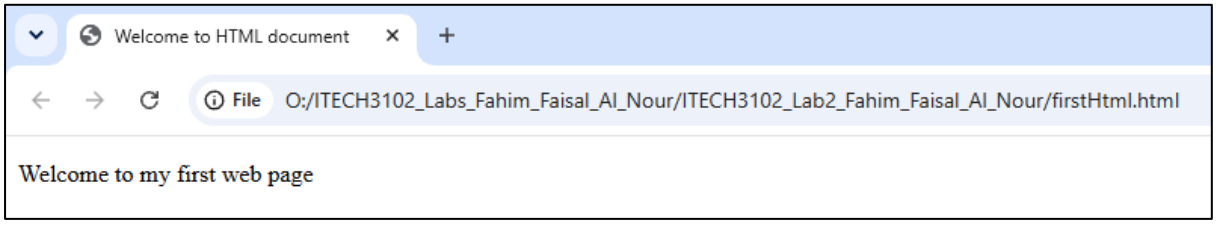


Figure 2.2: HTML file viewed in Google Chrome



2.2 Project 2: Using Browser's Built-in Developer Tools

Procedure: A second HTML file named second.html was created. This file included an unordered list (<ul>) containing items like "fevers", "chills", and "general malaise". The file was opened in Chrome, and the Developer Tools (DevTools) were launched by pressing F12. Under the Sources tab, the Workspace panel was selected, and the button Select folder was clicked to add the local project folder to the workspace. A new paragraph This text was added directly in the Chrome Developer Tools! was inserted into the code within DevTools. The file was saved using Ctrl+S, and the browser page was refreshed.

Result/Observation: After refreshing the browser, the newly added text successfully appeared on the web page, proving that the Chrome DevTools workspace was correctly linked and saving changes directly updated the local HTML file.

Figure 2.3: Before updated HTML page and Developer Tools Workspace

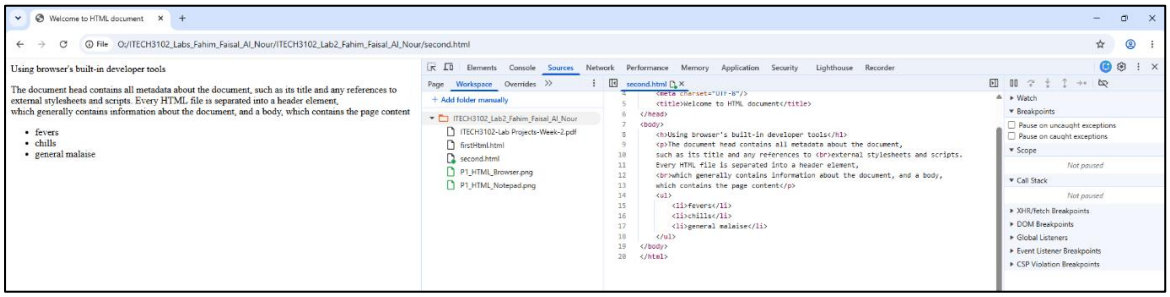
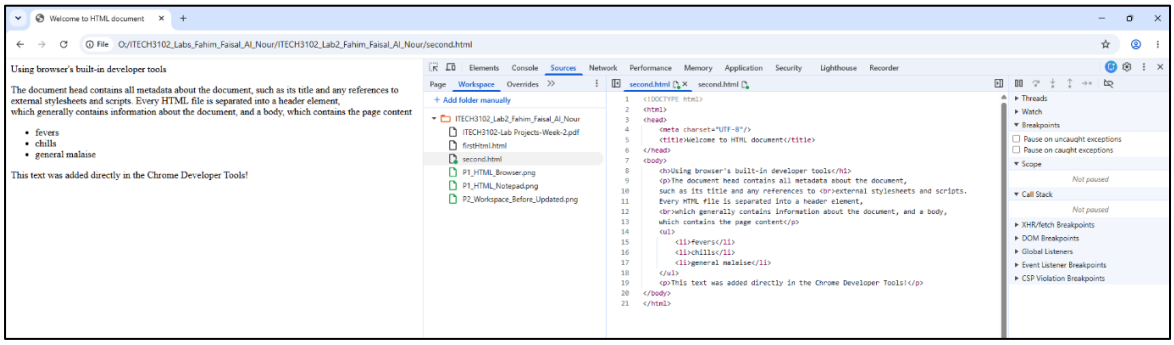


Figure 2.3: Updated HTML page and Developer Tools Workspace



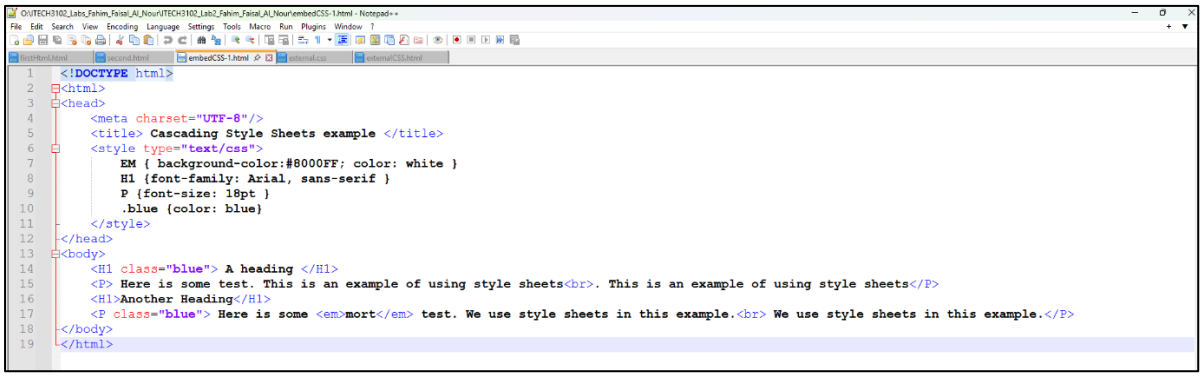
2.3 Project 3: Cascading Style Sheets (CSS)

2.3.1 Embedding CSS in HTML

Procedure: An HTML file named embedCSS-1.html was created. Inside the <head> section, a <style> tag was used to embed CSS rules directly. Styles were defined for the EM, H1, and P elements, along with a class named .blue.

Result/Observation: The file was opened in Chrome. The CSS was successfully applied to the HTML elements. The EM text had a purple background with white text, the H1 used the Arial font, and elements with the .blue class were rendered in blue color.

Figure 2.6: Embedded CSS rendered code



```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8"/>
    <title> Cascading Style Sheets example </title>
    <style type="text/css">
        EM { background-color:#8000FF; color: white }
        H1 {font-family: Arial, sans-serif }
        P {font-size: 18pt }
        .blue {color: blue}
    </style>
</head>
<body>
    <H1 class="blue"> A heading </H1>
    <P> Here is some test. This is an example of using style sheets<br>. This is an example of using style sheets</P>
    <H1>Another Heading</H1>
    <P class="blue"> Here is some <em>mort</em> test. We use style sheets in this example.<br> We use style sheets in this example.</P>
</body>
</html>
```

Figure 2.5: Embedded CSS rendered in Chrome



2.3.2 Using External CSS

Procedure: An external stylesheet named external.css was created in the same folder. It contained rules to style p elements with a blue background, white text, bold font weight, and a 24px font size. An HTML file named externalCSS.html was created. Inside the <head> section, the external CSS file was linked using the <link rel="stylesheet" href="external.css"> tag.

Result/Observation: The externalCSS.html file was opened in Chrome. The text paragraph was successfully displayed with a blue background and white text, confirming that the external stylesheet was linked and applied correctly.

Figure 2.6: External CSS file created in Notepad++

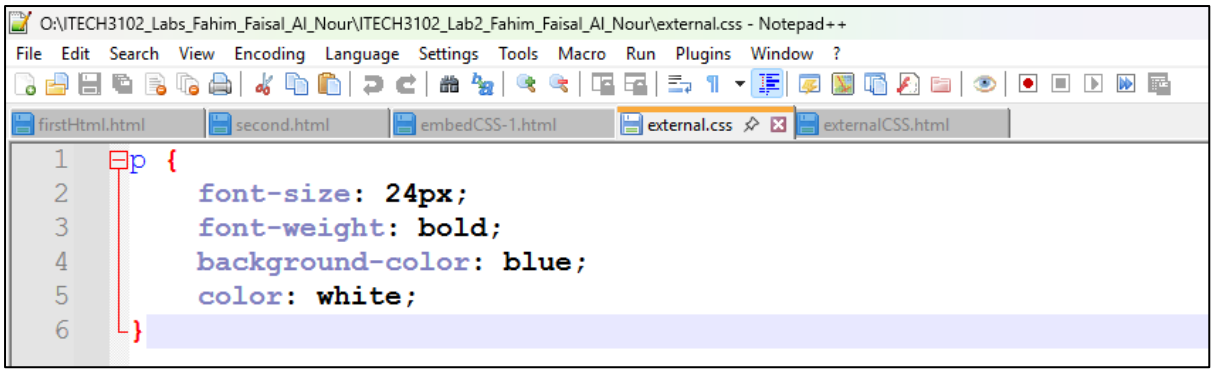
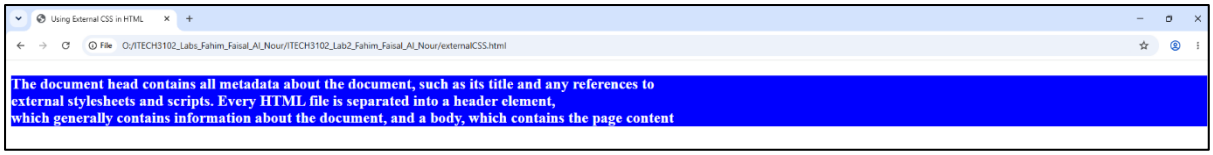


Figure 2.7: External CSS file successfully linked and rendered in Chrome



2.4 Project 4: Answering Questions

2.4.1 Comment on "Perception in Visualization"

The provided web resource by Dr. Christopher G. Healey at NC State University offers an in-depth analysis of how human visual perception operates and how it directly influences the design of effective data visualizations. The article explains that the human visual system is highly efficient at pre-attentively processing certain visual properties like color, motion, orientation, and texture without requiring conscious effort. By leveraging these inherent perceptual strengths, visualizations can be designed to instantly draw the viewer's attention to patterns, outliers, or correlations in complex datasets. I found the discussion on the limitations of human memory and attention particularly insightful, as it highlights why cognitive load must be minimized in visualization design. Furthermore, the strategic use of perceptual principles ensures that important information is not just seen, but also correctly interpreted and retained. This reading underscored for me that visualization is not merely about showing data, but about designing visual displays that align with the natural capabilities and limitations of the human brain.

2.4.2 Examples of Data Visualization Encoding Based on Perception Principles

The most common and highly effective encoding approach is the use of **position and length** to represent quantitative values, as seen in standard bar charts. According to pre-attentive processing principles, the human eye can accurately and rapidly compare lengths positioned along a shared baseline. This makes bar charts exceptionally perceptually accurate for comparing magnitudes.

Another excellent perceptual encoding method is the use of **color hue** to represent categorical data. Using distinct colors for different categories (e.g., the different boroughs in our Lab 1 dataset) acts as a powerful pre-attentive visual cue. It allows the audience to instantly distinguish between groups without needing to read text labels. This approach is particularly valuable when multiple categories need to be identified simultaneously, as color is processed very quickly and effectively by the human visual system, drastically reducing the cognitive effort required to understand the underlying structure of the data.

Chapter 3: JavaScript

3.1 Project 1: JavaScript

Procedure: Two methods of using JavaScript were explored. First, an HTML file named firstJava.html was created using Notepad++. The <script> tag was utilized within the document's body to write a welcome message using document.writeln(). This file was opened in Google Chrome to view the result. Second, the Chrome Developer Tools Console was opened. A line of JavaScript code, console.log(), was typed directly into the console to test immediate code execution.

Result/Observation: The Notepad approach successfully displayed the formatted welcome message in the browser. The Developer Tools Console approach instantly printed the string message to the console window, confirming that JavaScript can be executed directly in the browser environment without an external file.

Figure 3.1: JavaScript code written

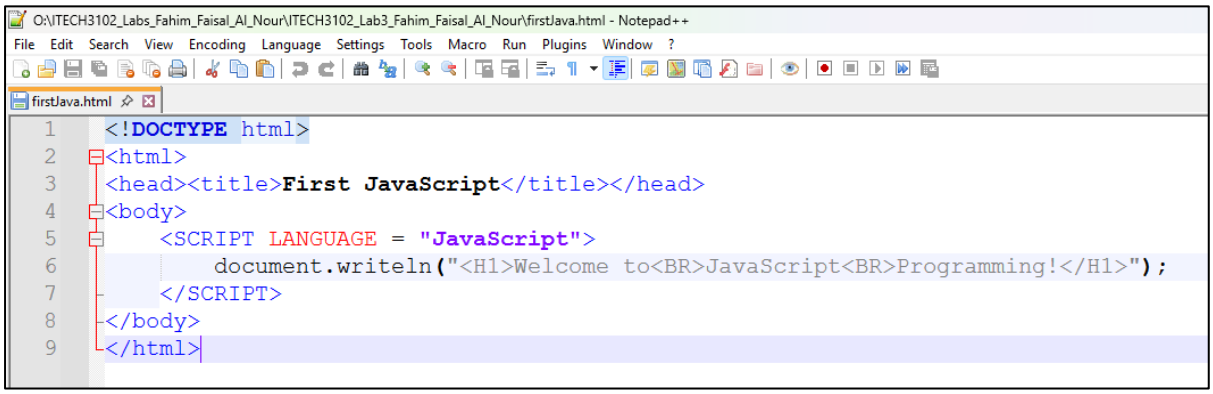
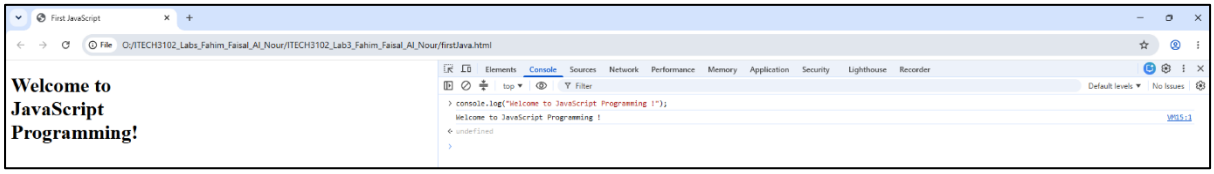


Figure 3.2: HTML file rendered in Chrome + Testing JavaScript in Chrome Developer Tools Console



3.2 Project 2: Variables, Arrays, Objects

Procedure: Using the Chrome Developer Tools Console, basic JavaScript data structures were tested. Variables were declared using the var keyword. Numeric and string arrays were defined using square brackets [], and their values were accessed using array indices. A custom JavaScript object was created using curly brackets {}, defining properties such as kind and color, and values were accessed using dot notation. An array of objects was created to demonstrate nesting. Finally, a separate HTML file was created to utilize the window.alert() method to display a multi-line alert dialog box.

Result/Observation: The console correctly returned the stored values when the variables and array indices were called. The dot notation successfully accessed the specific properties of the

object. The array of objects was correctly indexed. The window.alert() method successfully generated a pop-up dialog box displaying the text formatted with newline characters (\n).

Figure 3.3: Variables, Arrays, and Objects in Console

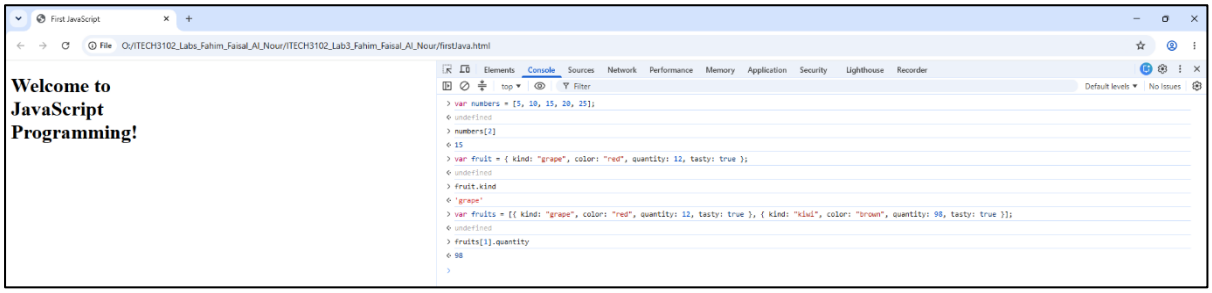


Figure 3.4: Alert dialog box code

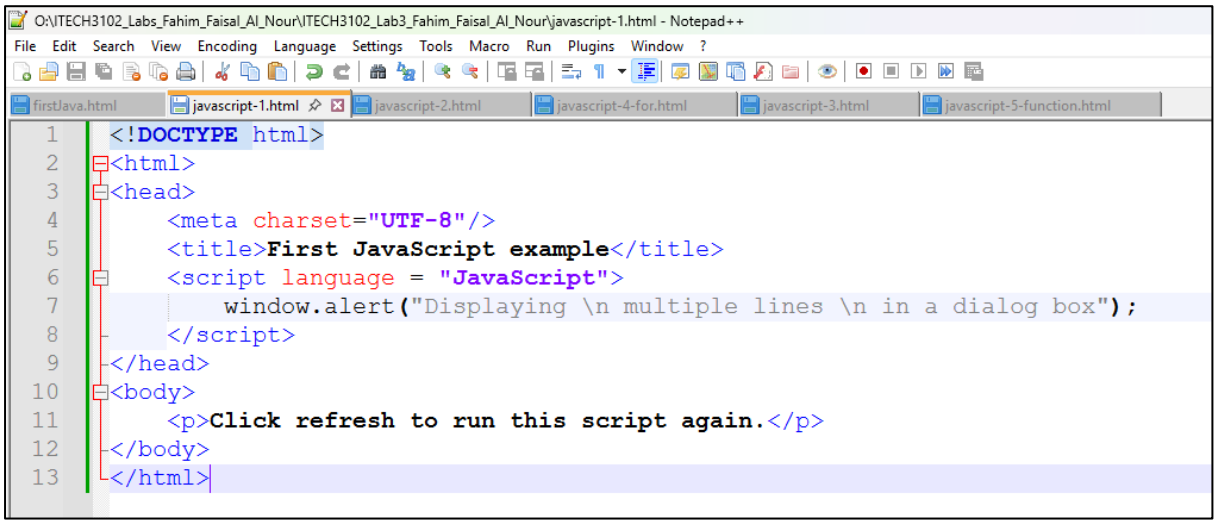


Figure 3.5: Alert dialog box triggered by JavaScript



3.3 Project 3: Basic control structures of JavaScript

3.3.1 Selection Structure (if-else)

Procedure: An HTML file was created to demonstrate an if-else selection structure. The window.prompt() function was used to accept user input. An if statement was implemented to check if the input grade was greater than or equal to 50.

Result/Observation: When a value of 40 was entered into the prompt, the program correctly evaluated the condition as false and executed the else block, displaying "Failed" and the corresponding message on the screen.

Figure 3.6: JavaScript if-else ***selection code***

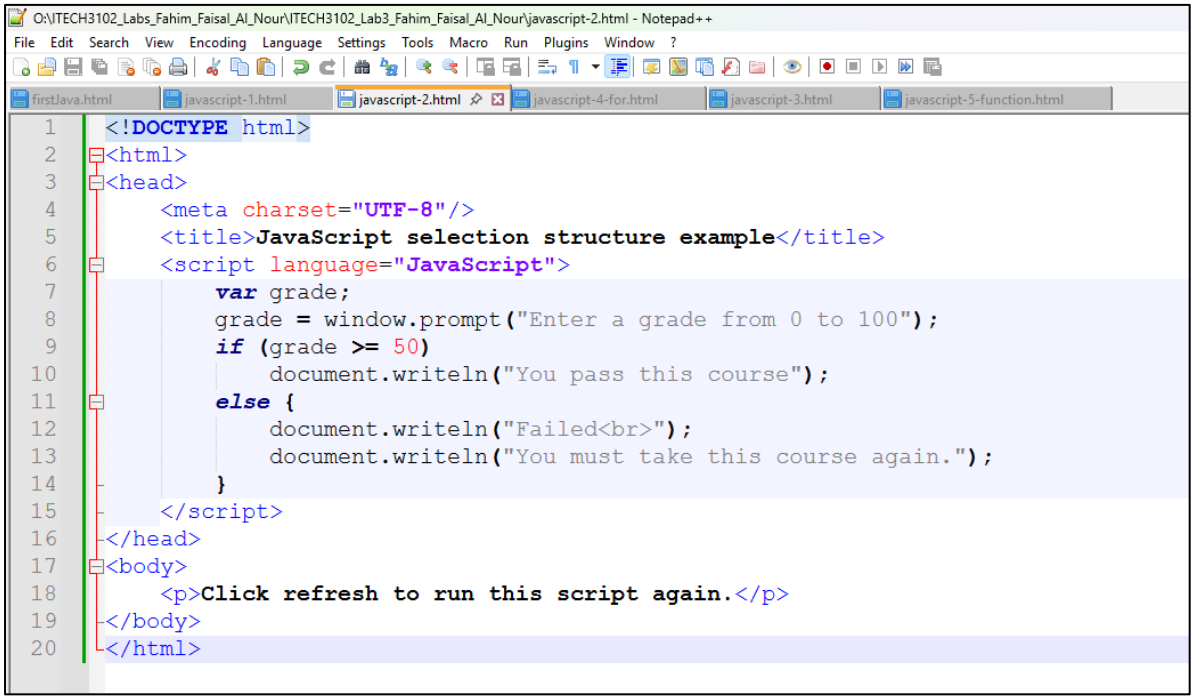


Figure 3.7: JavaScript if-else ***selection structure input box***

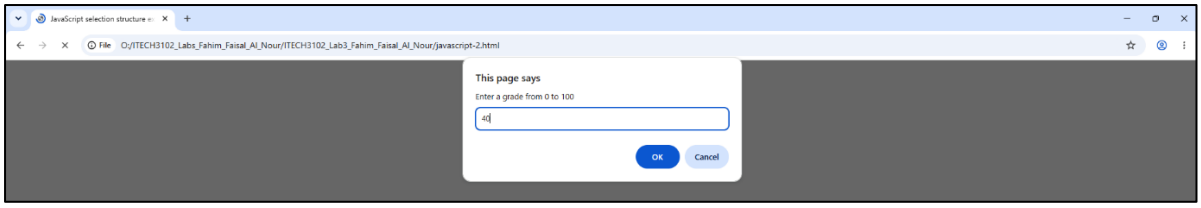
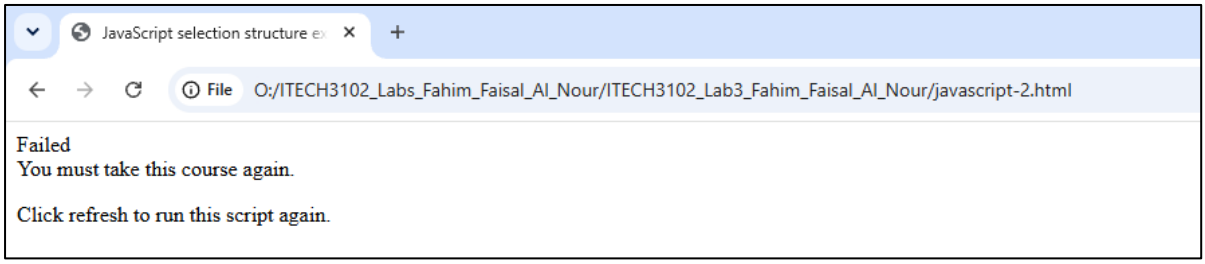


Figure 3.8: JavaScript if-else ***selection structure executed***



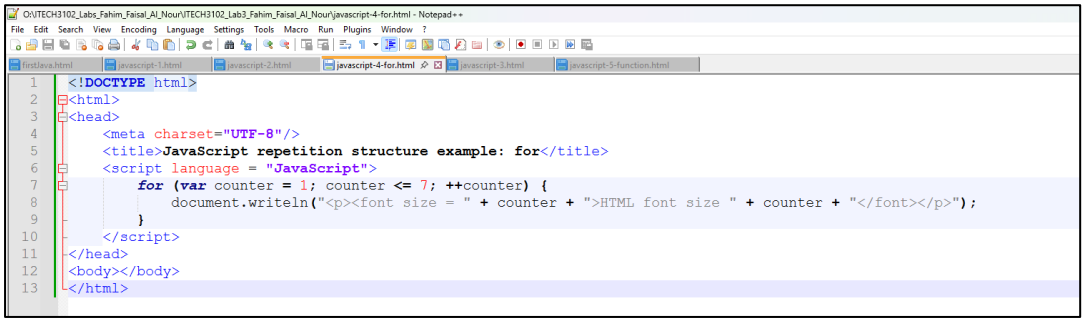
3.3.2 Repetition Structures (for and while)

Procedure: Two HTML files were created to explore JavaScript's repetition structures. The first script utilized a for loop to iterate a counter from 1 to 7, dynamically generating HTML <font> tags to display text with increasing sizes. The second script utilized a while loop, where a counter was initialized outside the loop, incremented inside the loop body, and terminated when the condition was no longer met.

Result/Observation: The for loop successfully generated 7 lines of text, each with a progressively larger font size. The while loop generated 6 lines of text with increasing font sizes,

demonstrating how both control structures can be used to perform repetitive tasks and manipulate the HTML document.

Figure 3.9: Code of the* for *loop repetition structure



```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8"/>
    <title>JavaScript repetition structure example: for</title>
    <script language = "JavaScript">
        for (var counter = 1; counter <= 7; ++counter) {
            document.writeln("<p><font size = " + counter + ">HTML font size " + counter + "</font></p>");
        }
    </script>
</head>
<body></body>
</html>
```

Figure 3.10: Output of the* for *loop repetition structure

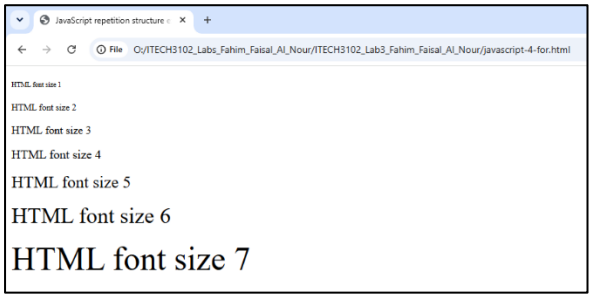
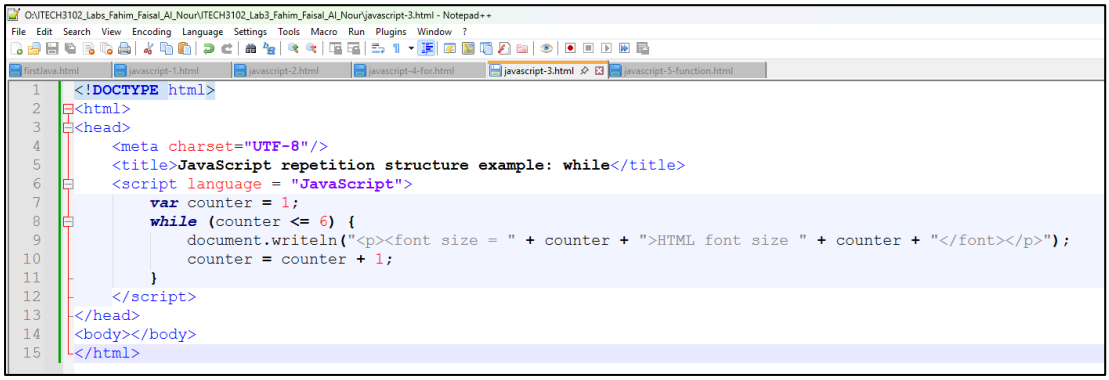
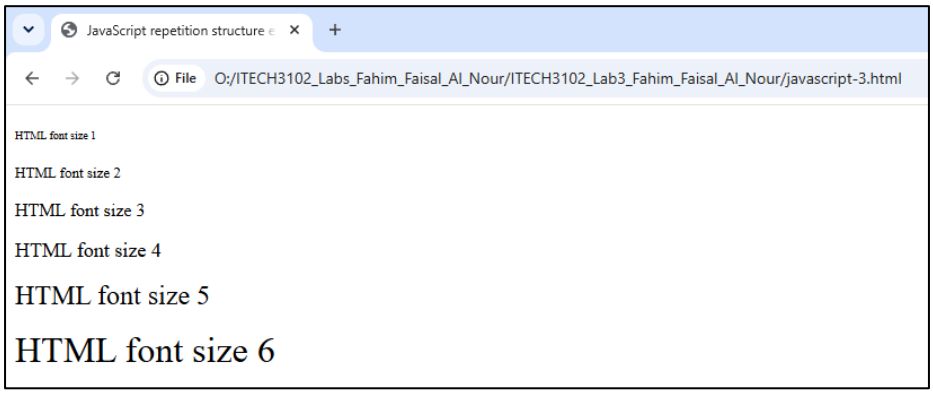


Figure 3.11: Code of the* while *loop repetition structure



```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8"/>
    <title>JavaScript repetition structure example: while</title>
    <script language = "JavaScript">
        var counter = 1;
        while (counter <= 6) {
            document.writeln("<p><font size = " + counter + ">HTML font size " + counter + "</font></p>");
            counter = counter + 1;
        }
    </script>
</head>
<body></body>
</html>
```

Figure 3.12: Output of the* while *loop repetition structure



3.3.3 User-defined Function

Procedure: A custom function named square() was defined using the function keyword, which accepts a single argument and returns the square of that number. A for loop was then written to call this square() function for numbers 1 through 10, outputting the results to the browser.
Result/Observation: The page successfully displayed the squares of the numbers 1 through 10, proving that the custom function was correctly defined, passed arguments, and returned computed values to the main script.

Figure 3.13: Code of the user-defined square() ***function***

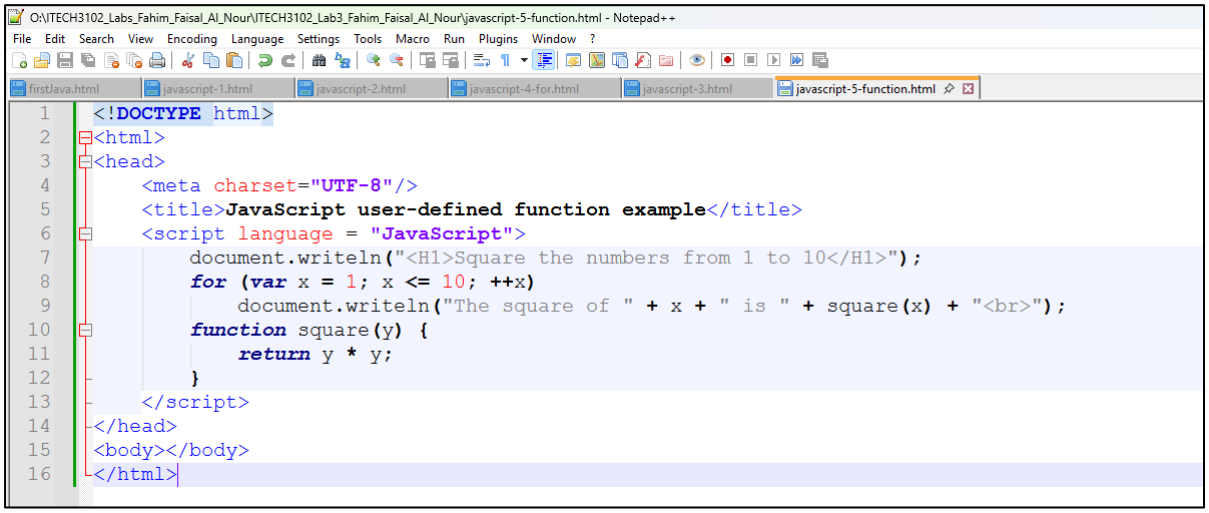
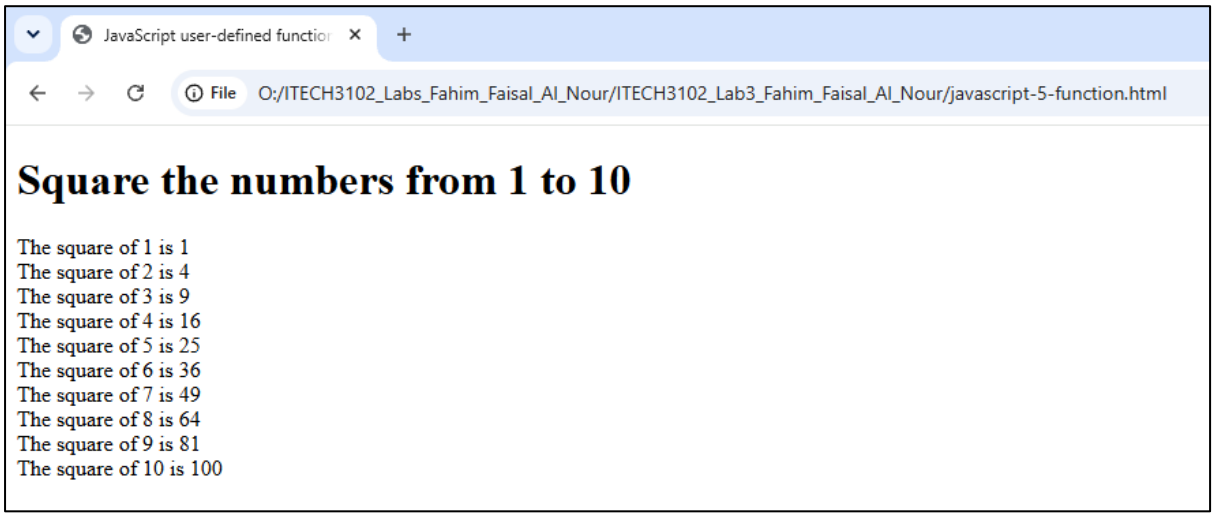


Figure 3.14: Output of the user-defined square() ***function***



3.4 Project 4: Answering Questions

3.4.1 What is data abstraction? Give one or two examples of data abstraction.

Data abstraction is a fundamental concept in computer science and programming that involves simplifying complex systems by hiding unnecessary implementation details and exposing only the essential functionalities to the user. It allows a programmer to interact with an object at a

high level without needing to understand its intricate underlying code or internal mechanisms. This reduces complexity and increases modularity.

- **Example 1:** Driving a car. A driver uses the steering wheel, accelerator, and brake pedals. The driver does not need to know the complex mechanics of how the fuel injection system works or how the engine pistons move. The user interface (the pedals and wheel) abstracts the complexity of the engine.
- **Example 2:** Using a smartphone. A user interacts with a graphical interface (screen, apps, touch gestures) to make a call or send a message. They do not need to know the electrical signals, transistor logic, or operating system code that runs in the background to process these actions. The app interface abstracts the hardware complexity.

3.4.2 Why do we need to derive data from raw data?

Raw data is often messy, unstructured, incomplete, or too granular to be directly useful for analysis or decision-making. Therefore, deriving or transforming raw data into processed data is necessary for several reasons:

- **Data Cleaning and Noise Reduction:** Raw data frequently contains errors, missing values, or outliers. Deriving data allows us to filter out these anomalies and replace them with consistent, high-quality values.
- **Aggregation and Summarization:** Raw data is often too detailed (e.g., millions of individual sales transactions). Deriving aggregated data (such as total sales per month, or average revenue per customer) simplifies the dataset and makes it easier for decision-makers to identify long-term trends quickly.
- **Creating Meaningful Features:** Raw data may lack necessary context. By deriving new attributes from raw data (e.g., calculating "Profit" by subtracting "Cost" from "Revenue", or deriving "Age" from a "Date of Birth" field), we create dimensions and measures that have higher analytical value.
- **Formatting for Visualization:** Visualization tools like Tableau or D3 often require data in specific formats (e.g., geospatial coordinates, specific date formats). Deriving data ensures the dataset is structured in a way that is compatible with the visualization platform.

Chapter 4: Scalable Vector Graphics (SVG) and D3

4.1 Project 1: Scalable Vector Graphics (SVG)

Procedure: In this experiment, I explored the use of Scalable Vector Graphics (SVG) to draw basic 2D shapes directly inside an HTML document. I created several distinct HTML files using Notepad++. In each file, I opened an <svg> canvas with a specified width and height. I then utilized different SVG shape elements: <circle> (using cx, cy, and r for center and radius), <rect> (using x, y, width, and height), <ellipse> (using cx, cy, rx, and ry for the two radii), <line> (using x1, y1, x2, y2, and a required stroke), and <text> (using x, y, and font styling attributes). All files were saved within my project folder and opened using Google Chrome.

Result/Observation: I successfully rendered all the geometric shapes in the web browser. The SVG system correctly interpreted the coordinate system where (0, 0) is the top-left corner. The output confirmed that SVG provides a powerful, vector-based approach to creating sharp, scalable graphics directly within a browser environment.

Figure 4.1: HTML code for drawing an SVG circle

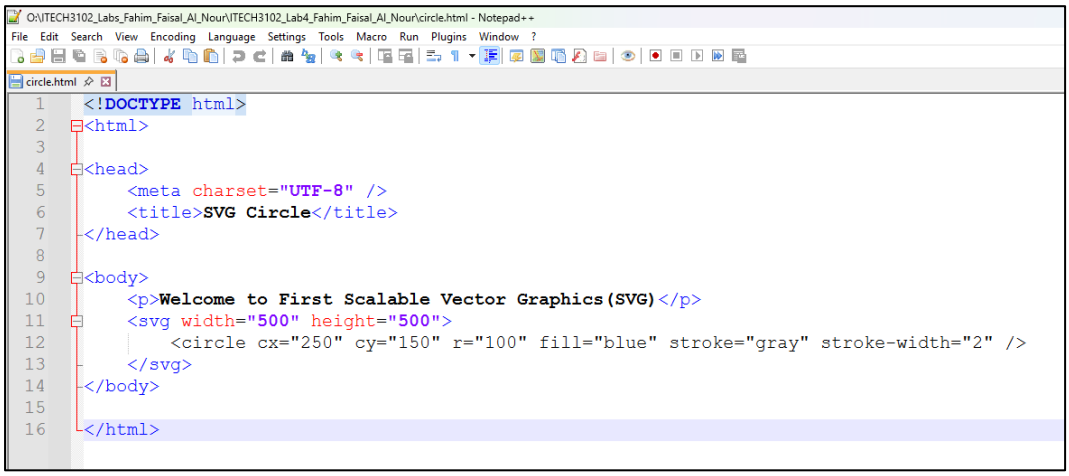


Figure 4.2: SVG Circle rendered in Chrome

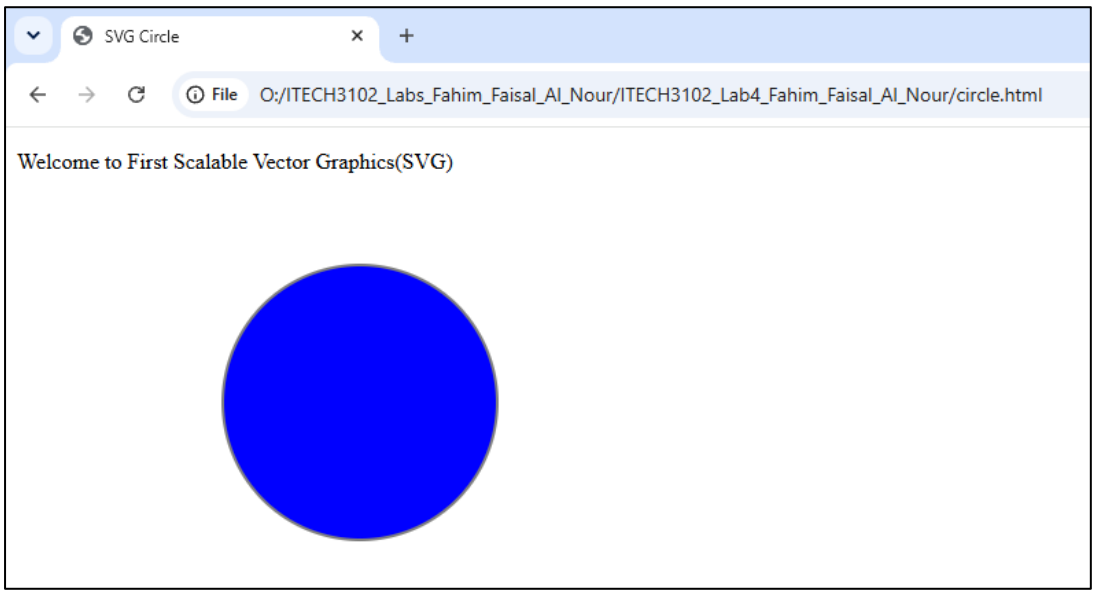


Figure 4.3: HTML code for drawing an SVG rectangle

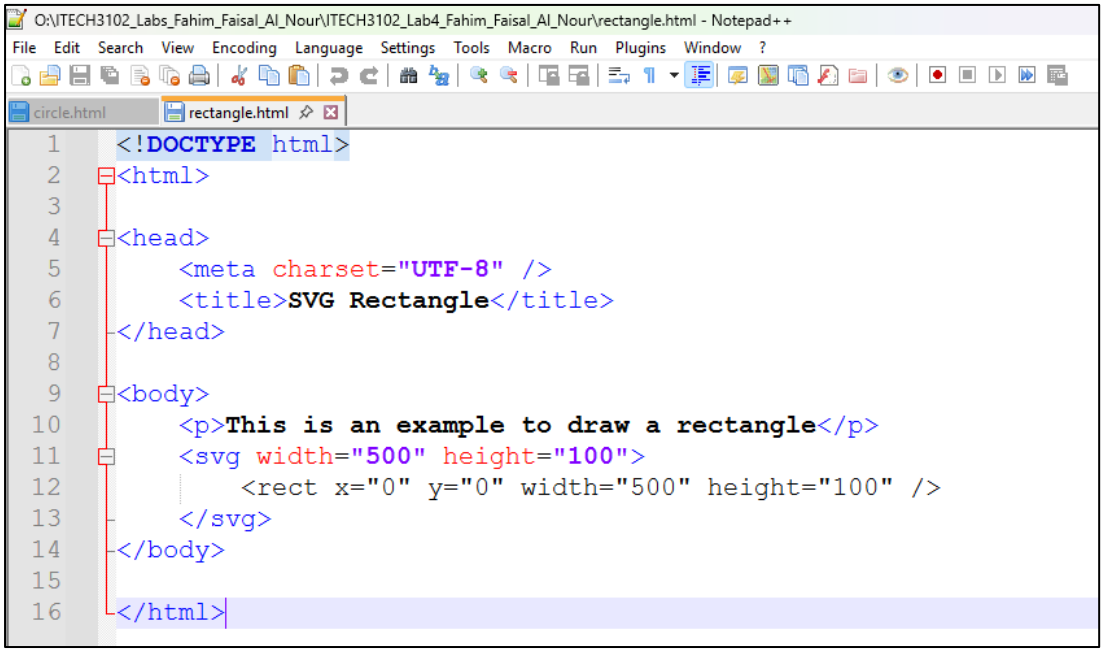


Figure 4.4: SVG Rectangle rendered in Chrome

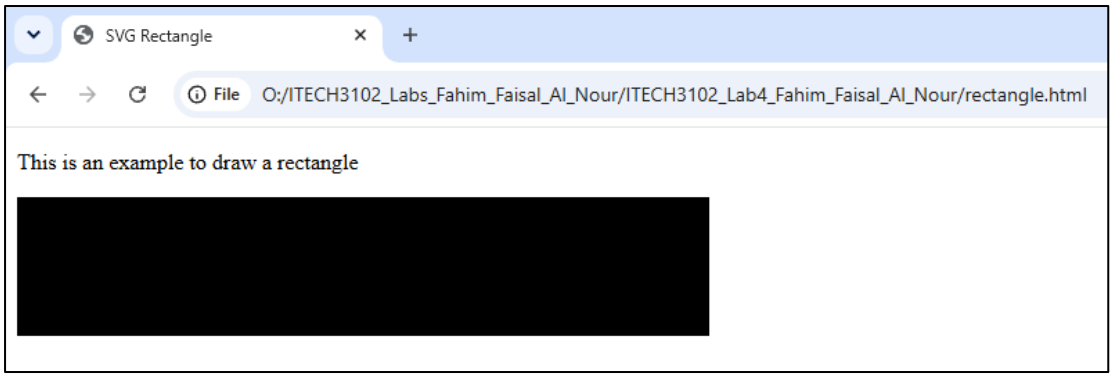


Figure 4.5: HTML code for drawing an SVG ellipse

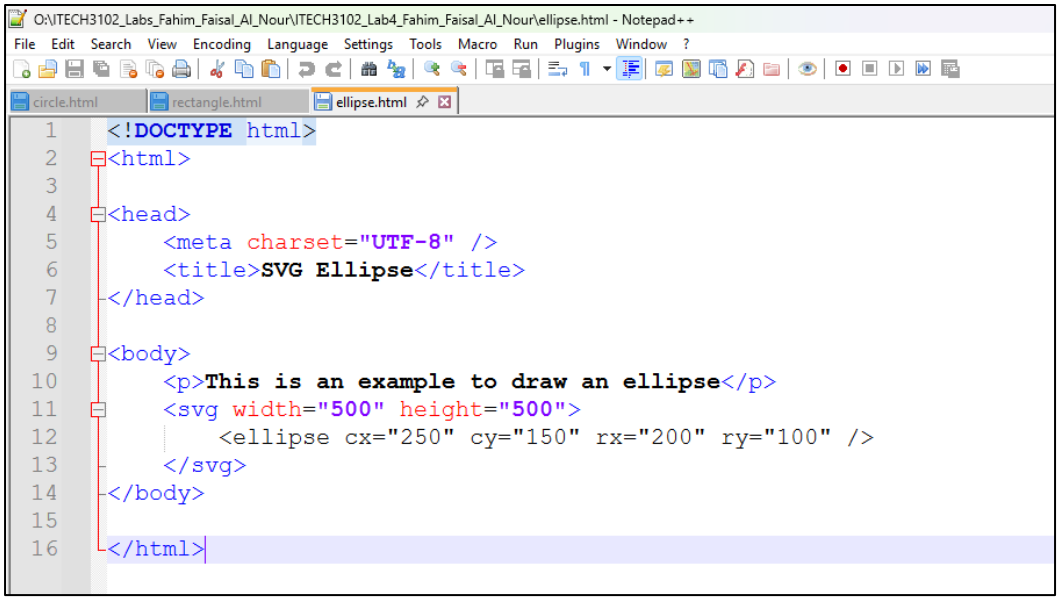


Figure 4.6: SVG Ellipse rendered in Chrome



Figure 4.7: HTML code for drawing an SVG line

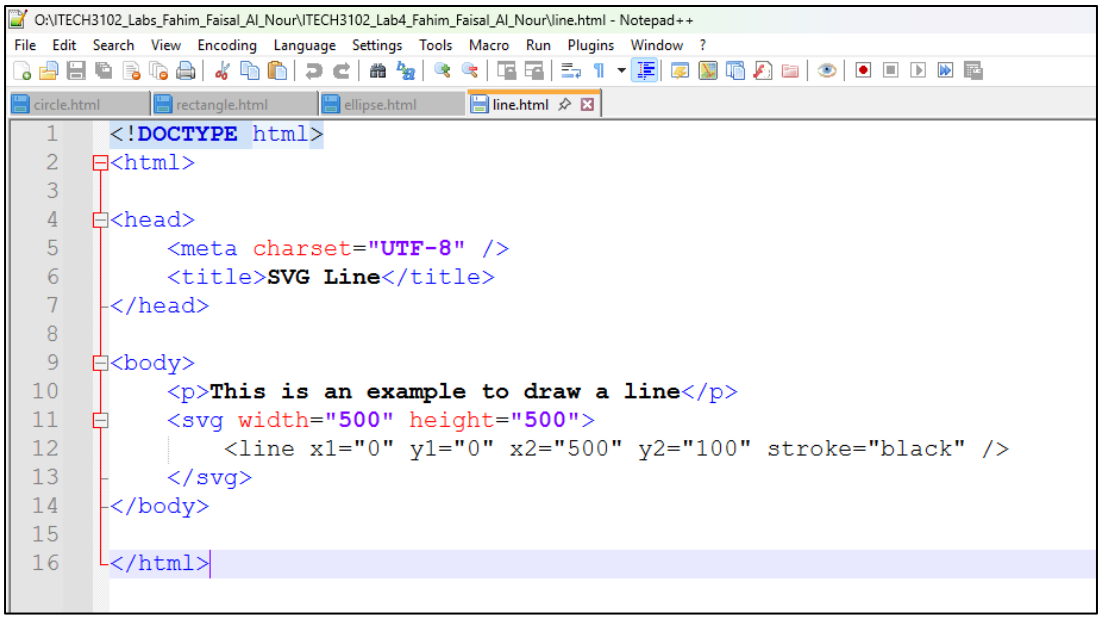


Figure 4.8: SVG Line rendered in Chrome

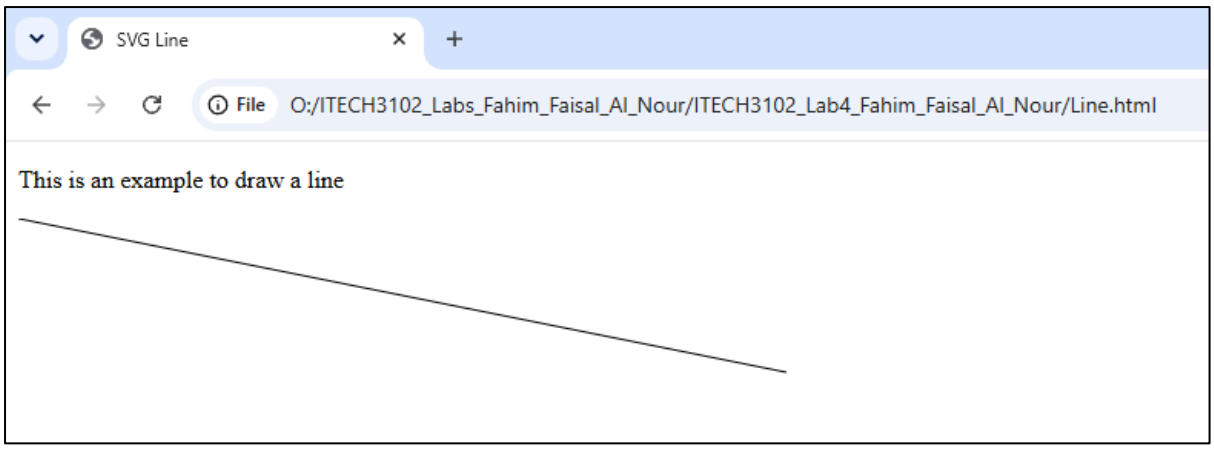


Figure 4.9: HTML code for drawing SVG text

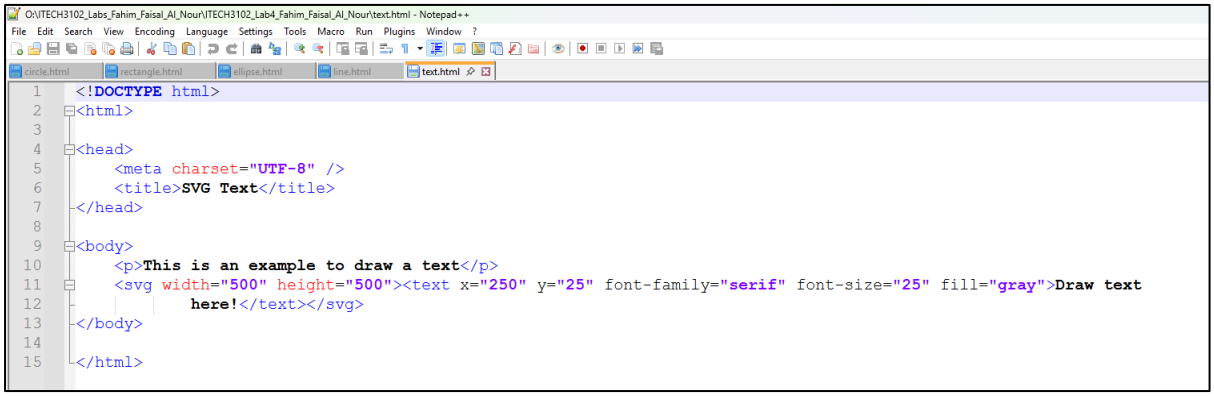
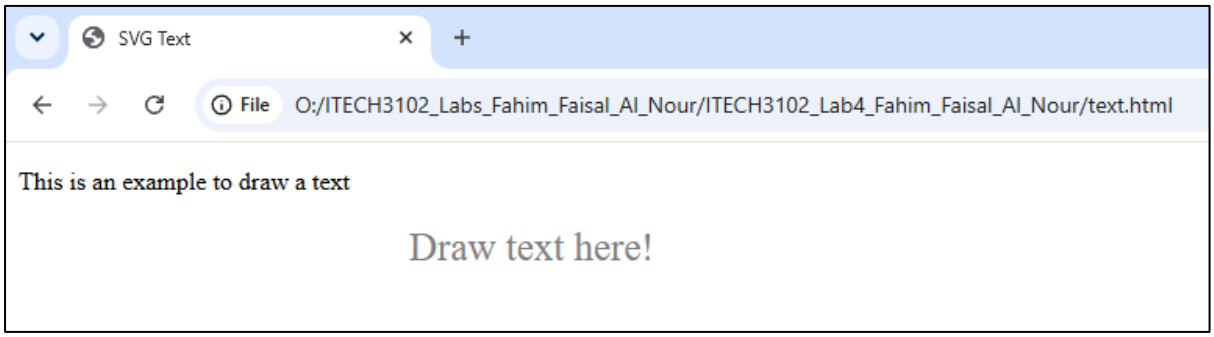


Figure 4.10: SVG Text rendered in Chrome



4.2 Project 2: Styling SVG Elements, Layering and Transparency

Procedure: To understand how SVG handles styling and rendering order, I created an rgba.html file. I added five overlapping circles using the <circle> element, placed sequentially from left to right. Instead of using standard solid colors, I applied the rgba() color function to the fill attribute. I varied the alpha (transparency) parameter from 1.0 (fully opaque) on the first circle down to 0.1 (nearly transparent) on the last circle. The circles were drawn in the order they appeared in the HTML code to observe the layering effect.

Result/Observation: Upon opening the HTML file in Chrome, I observed that SVG does not use a CSS z-index for depth; instead, it adheres strictly to the document's drawing order. The first circle listed in the code was drawn at the "bottom" of the overlapping stack, while the last circle was drawn on the "top". The varying alpha transparency levels allowed the colors to blend visually, producing a smooth gradient effect. This proved that transparency and layering are powerful tools for visually reducing data occlusion in complex visualizations.

Figure 4.11: Code demonstrating SVG layering and RGBA transparency

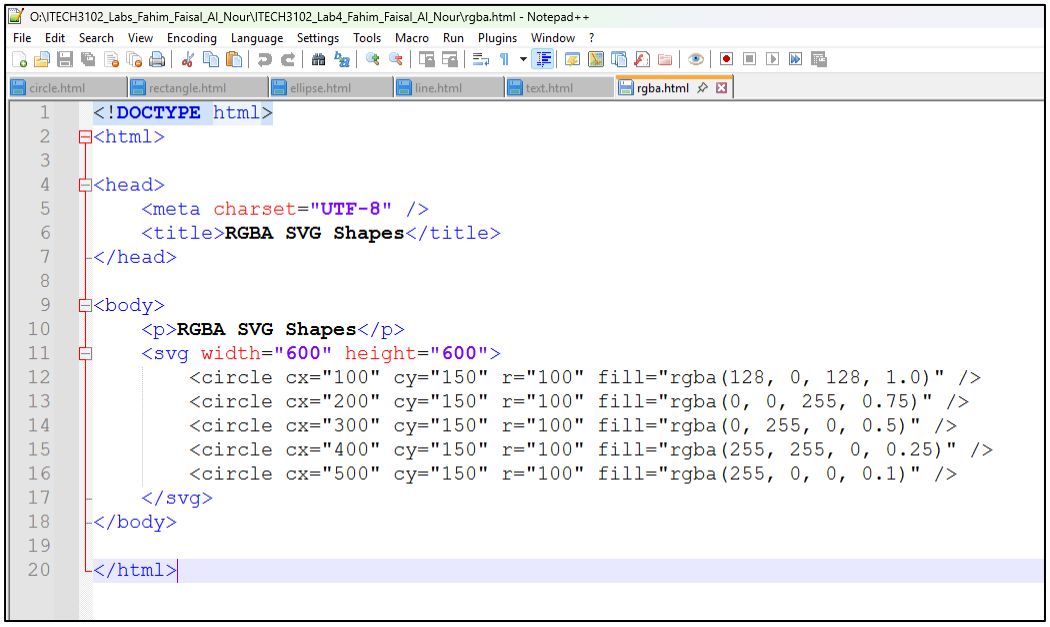
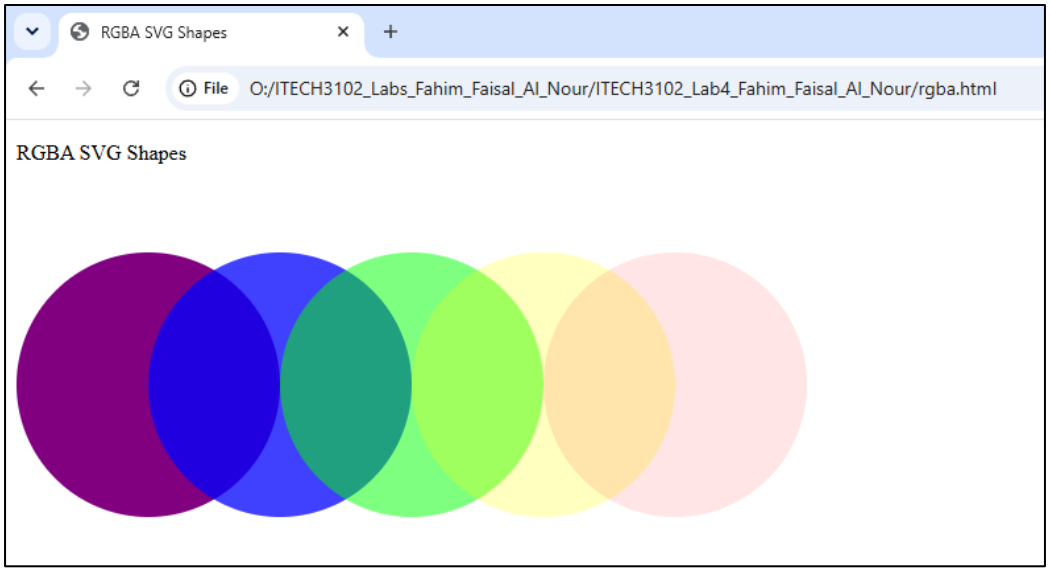


Figure 4.12: Overlapping semi-transparent SVG circles rendered in Chrome



4.3 Project 3: Setting up a D3 Development Environment

Procedure: Setting up the D3 environment required configuring a local HTTP server, as modern browsers block D3 from loading external .js libraries directly from the local file:/// protocol for security reasons. First, I created a dedicated project folder and downloaded the d3.js library, placing it in the same directory as my HTML files. I used index.html as a template to verify the library reference.

Next, I installed Anaconda Distribution on my computer. I opened the **Anaconda Prompt** terminal and ran the conda update --all command to ensure all underlying Python packages were up to date. Using the cd (change directory) command in the terminal, I navigated directly into my Lab 4 project folder. Finally, I executed the Python command python -m

http.server 8888, which spun up a local web server. To test the setup, I created a firstserver.html file that contained simple D3 code to append a new paragraph to the DOM.

Result/Observation: The server started successfully on port 8888. I opened my browser and navigated to http://localhost:8888/firstserver.html. The D3 code successfully executed, displaying the initial HTML text and dynamically adding the text "New paragraph!" to the page. This confirmed that my local Python server was correctly hosting the files and the D3 library was successfully linked.

Figure 4.13: Project folder containing d3.js ***and HTML files***

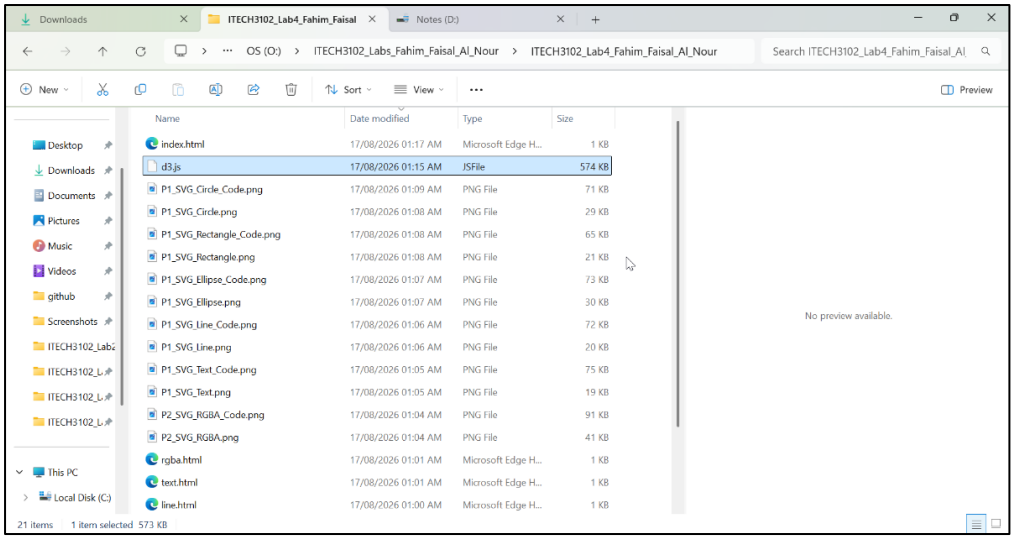


Figure 4.14: Anaconda Prompt updating packages

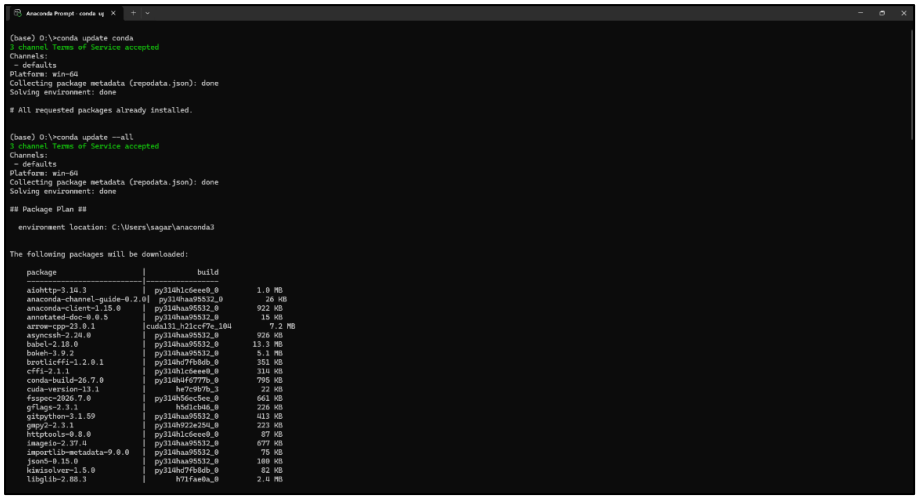
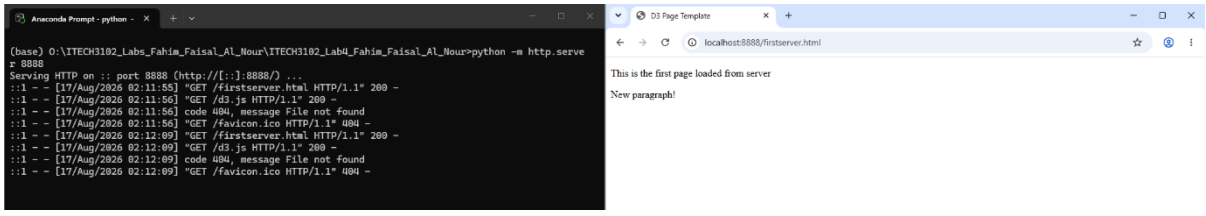


Figure 4.15: HTML page loaded successfully through the local server at localhost:8888



4.4 Project 4: Answering Questions

4.4.1 Based on concepts of the marks and channels, provide some examples of appropriate and inappropriate encoding choices.

Based on the principles of visual perception (as described in Munzner's Chapter 5), appropriate encoding aligns the channel's expressive power with the data type. For example, using **position on a common scale (x/y axes)** for quantitative data is an extremely appropriate encoding, as humans are highly accurate at perceiving variations in position. Similarly, using **color hue** for categorical data (e.g., distinguishing Country A from Country B) is highly appropriate because the human eye quickly processes color differences.

On the other hand, **color hue** is an inappropriate channel for quantitative data. A rainbow spectrum does not convey a natural order, making it difficult for the brain to judge which value is numerically higher or lower without constantly checking the legend. Additionally, using **area or size** is often an inappropriate channel for comparing small differences in numerical values. Humans perceive size on a logarithmic scale, meaning small differences (e.g., values of 50 vs. 55) would be visually indistinguishable, leading to inaccurate data interpretation.

4.4.2 Take an example of good visualizations (the graph presented in the topic 1, Napoleon's Army During the 1812, Visualization created by Charles Joseph Minard), and explain what marks and channel are used in the visualization, and discuss why they are used for?

Charles Joseph Minard's visualization of Napoleon's 1812 Russian campaign is widely considered a masterpiece of data storytelling because it encodes **six different variables** into a single, flowing 2D image.

It primarily uses **lines and areas** as marks. The thick, flowing bands represent the **path** and **direction** of the army.

Regarding the **channels** used: **Position (x and y coordinates)** is used to precisely map the retreat and advance routes onto geographic longitude and latitude. The **size (thickness/area) of the band** is used to encode the quantitative variable of the **army's size** (number of soldiers). As the band narrows, the viewer instantly perceives the catastrophic loss of human life.

Finally, **color hue** is used categorically to represent the **direction** of travel: tan/gold colors represent the advance into Russia, while the stark black represents the brutal retreat.

These channels were combined because each one aligns perfectly with the data it represents. Position is the most accurate channel for geography, size is excellent for expressing severe magnitude changes, and color provides an instant categorical distinction. This effective combination allows the human brain to grasp a complex historical narrative instantly without overwhelming the viewer with text.
